Agentic AI, Assignment 2: GigBid

R&D Report (AI-4016)
FAST School of Computing, Fall 2026

Student details

Full name	Abdul Basit
Roll number	22i-0866
Class / section	Agentic AI - B
University email	i220866@nu.edu.pk
GitHub username	joe1o

Project

Project name	GigBid
Domain	Multi-agent freelancer copilot: finds jobs, flags scams, writes grounded proposals, a human approves every send
GitHub repository URL	https://github.com/joe1o/gigbid
Instructor	Muhammad Owais Idrees

Live links

Live app	https://gigbid-vkhw.onrender.com
Reviewer login	Shared in the private submission comment
Health endpoint (GET)	https://gigbid-vkhw.onrender.com/api/ping
API documentation	https://gigbid-vkhw.onrender.com/docs
Public traces	Three LangSmith traces, listed in Appendix B and the README

Deployment and models

Hosting provider	Render (free plan, Singapore region, one process)
Database	Neon Postgres (Singapore): checkpoints, memory store, users
Language models	gpt-oss-120b for the Writer and Critic, gpt-oss-20b for the other model calls; open weights, served by Groq
Retrieval models	BAAI/bge-small-en-v1.5 embeddings and ms-marco-MiniLM-L-6-v2 reranker, on CPU
Tracing	LangSmith

Summary. Every number in this report cites a file in `evals/results/`, a public trace or a commit; the data is synthetic (`docs/DATA_CARD.md`). On 51 dev jobs GigBid reached 0.957 bid or skip F1, caught all 6 scams, priced 84.8% of bids inside the fair band and cut unsupported claims from 0.152 to 0 per proposal, at $0.0015 per proposal and a p95 of 116 s per job (`decision_eval.json`). The main negative result: the simplest retriever (one dense query over the whole post) beat every advanced variant, and our pre-registered hypothesis was rejected.

1. Problem and constraints

A freelancer on a marketplace such as Upwork spends hours each week reading posts, ignoring scams, deciding what to bid on, pricing, and writing proposals. Each bid costs Connects (paid credits), so a bad bid wastes money as well as time. GigBid automates the reading, judging and drafting, and leaves every decision that commits the user to a human.

Never automated. Sending a proposal, sending a client message and spending Connects. The bid graph stops at a LangGraph `interrupt()` and waits for approve, edit or reject; the conversation graph does the same before every reply. Nothing reaches the marketplace until the user clicks.

Other constraints. Open models only on the required profile; act within platform rules (no contact details, no off-platform talk, no guarantees, required statements); never claim experience the portfolio cannot show; poll like a person, not a scraper (every 2 hours on GigSim, 6 hours live); keep running on a free host with 512 MB of memory.

Budgets (enforced in `agent/config.yaml` and checked in CI by `evals/thresholds.yaml`):

Budget	Declared	Measured on dev (`decision_eval.json`)
Graph steps per job	at most 30, else "needs attention"	never exceeded
Tokens per job	at most 40,000, else "needs attention"	5,403 on average
Latency per job analysed, p95	120 s	116.5 s (p50 20.6 s)
Cost per job analysed	$0.005	$0.0011
Cost per proposal	$0.01	$0.0015
Unsupported claims after verification (CI gate)	0	0

2. Architecture

```
React + Vite + TypeScript  --HTTP + SSE-->  FastAPI (graph runs are background tasks)
        |                                        |
        |                     LangGraph bid graph + conversation graph
        |                     Postgres checkpointer + store (Neon)
        |                                        |  MCP over Streamable HTTP, bearer auth,
        |                                        |  per-agent tool allowlists
        |        Portfolio MCP (4 tools)   Marketplace MCP (12 tools)   Workspace MCP (3 tools)
        |        RAG over the portfolio    port -> GigSim adapter       calendar, email drafts
        |                                       -> Upwork adapter (stub)
```

Bid graph (one thread per job, `agent/graphs/`, rendered in `docs/images/bid_graph.mmd`): `intake` fans out with `Send` to **Job Analyst**, **Risk Assessor** and **Fit Matcher** in parallel; their outputs join at **Decide** (bid, watch or skip). A bid goes to **Pricing**, the **Writer**, **claim verification**, and the **Critic**, which can send the draft back at most twice. The **Compliance Guard** then enforces the policy pack, and `prepare_approval` builds a dry-run preview (exact payload and Connects cost) before the `approval` interrupt. Approve submits, edit re-runs compliance and verification, reject ends the thread with the reason.

Conversation graph (one thread per client conversation): load the thread, classify intent (question, negotiation, interview, scam signal), plan with rules, draft, check compliance, and interrupt for approval before sending.

Why each agent exists.

Agent	Reason	Model
Scout	find new jobs on a schedule, drop duplicates and reposts by fingerprint	none
Job Analyst	turn a messy post into a typed `JobSpec`; flag injected instructions	20b
Risk Assessor	protect Connects and the user from scams and bad clients	rules + 20b
Fit Matcher	prove or fail to prove each requirement from the portfolio (RAG V8)	none
Decide, Pricing	bid or skip with reasons; price inside budget, above floor, from market stats	rules
Writer	cover letter and screening answers from retrieved evidence only	120b
Claim verification	remove sentences the evidence does not support	reranker
Critic	grounding, personalisation, length, screening answers; bounded loop	rules + 120b
Compliance Guard	contact details, guarantees, off-platform talk, required statements	rules
Client Liaison	classify replies, draft answers or interview slots	20b + 120b

Four agents use no model at all: they are rules, which are cheaper, faster and testable (`docs/adr/004-rules-vs-llm.md`). Agents hand off Pydantic objects (`agent/handoffs.py`), not text.

Marketplace port. Agents never call GigSim directly. The Marketplace MCP server calls one interface with two adapters: GigSim (used everywhere) and Upwork live (a stub behind the same port, selected by `MARKETPLACE_MODE`). Write tools take idempotency keys and support `dry_run`.

State, trace and memory are kept apart. *State* is what one job needs now (`agent/state.py`, checkpointed after every step, reducers on parallel writes). The *execution trace* is every tool call with arguments and result, in the MCP audit log (PII masked) and LangSmith. *Long-term memory* holds only promoted knowledge in the LangGraph store (`agent/memory/promotion_rules.py`): a style rule is promoted after it appears in 2 separate user edits, and an outcome summary for a category after 3 results. Raw tool output never goes straight into memory.

Stores. Neon Postgres: checkpoints, store, users, ledger. SQLite: GigSim's own data and the marketplace draft table. NumPy vectors per persona: the RAG index, cached by content hash.

Durability. A paused proposal survives a restart: trace 3 shows a brand-new runtime resuming a paused thread and submitting once (https://eu.smith.langchain.com/public/b6c10839-f9be-4e47-be7a-cfea81347f75/r). If the marketplace is down, the job pauses with status `queued`, the UI shows a banner, and a recovery loop resumes it (`scripts/degraded_mode_check.py`).

3. R&D question A: tool granularity

Hypothesis. Small tools with precise schemas give the model higher argument accuracy than a few large tools with an `action` field, at a higher token cost; rewriting descriptions adds little once names and schemas are clear.

Setup. 40 tasks with a known correct call (`evals/datasets/tool_tasks.json`), gpt-oss-20b, three tool menus: *fine* (our 19 shipped tools), *coarse* (3 tools, one per server, with an `action` argument), *rewritten* (fine tools with "use when" descriptions and an example).

Menu	Tool selection	Argument accuracy	Tokens per task
Fine (shipped)	97.5%	97.5%	1,656
Coarse	82.5%	55.0%	688
Rewritten descriptions	97.5%	97.5%	1,680

Source: `rnd_a_tools.json`.

Conclusion. Supported. Coarse tools cost 2.4 times fewer tokens but lost 15 points of selection and 42.5 points of argument accuracy: the model put arguments at the wrong level or sent the wrong `action`, and 4 coarse calls failed schema validation outright. Rewriting descriptions changed nothing; the only miss in both fine menus is one ambiguous task ("past proposals that won mention churn"), where `list_proposals` is a defensible answer. We keep the fine tools.

4. R&D question B: retrieval, claim verification and the shipped choice

Hypotheses were committed before any retrieval result (`evals/HYPOTHESES.md`, commit `224b5d3` before `6442d25`). H-B1: requirement decomposition plus hybrid BM25 and dense retrieval raises Recall@5 by 15 points over V0. H-B2:

the gain is largest on vague posts. H-B3: reranking adds under 5 points. H-B4: the corrective loop refuses evidence for poor-fit requirements.

Setup. 36 dev jobs with hand-marked gold evidence, three personas, 40-word chunks with 10 words of overlap, `BAAI/bge-small-en-v1.5` embeddings, `ms-marco-MiniLM-L-6-v2` reranker. Variants: V0 naive dense over the whole post; V1 one query per requirement; V2 = V1 + BM25 fused with RRF; V3 = V2 + cross-encoder rerank; V8 = V3 + a corrective LangGraph loop that rewrites the query or refuses when the reranker score is below -7.0.

Variant	Recall@5	MRR	nDCG@10	p95 ms	Unsupported claims / proposal	Judge	GigSim reply rate	Cost / proposal
V0 naive dense (shipped)	**0.898**	**1.000**	**0.955**	**18**	0.152 before, 0 after	4.6 / 5	60.6%	$0.0015
V1 decomposition	0.803	0.857	0.847	40				
V2 hybrid (RRF)	0.836	0.917	0.851	44				
V3 rerank	0.810	0.794	0.783	1,268				
V8 corrective	0.854	0.884	0.854	1,678				

Sources: `rag_eval.json` (retrieval), `decision_eval.json` (proposal columns, V0 only), `judge_calibration.json`. Proposal-level columns were measured only for the shipped variant: each full proposal run uses most of a day of the free provider quota.

Ablation (remove one component from V8): without decomposition 0.944, without hybrid 0.854 (no change), without rerank 0.836, without the corrective loop 0.810. Decomposition is the component that hurts; the corrective loop adds 4.4 points.

By persona and post style (Recall@5). V0: ML engineer 0.958, data analyst 0.875, web developer 0.861; detailed 0.897, vague 0.900. V1 on vague posts: 0.733, its worst slice.

Conclusions. H-B1 rejected: decomposition plus hybrid *lost* 6.2 points (V2 0.836 vs V0 0.898). H-B2 rejected: decomposition did worst on vague posts, where the extracted sub-queries are short (often one skill name) and likely match generic chunks. H-B3 rejected in a different way than expected: reranking lowered both Recall@5 and MRR. A likely reason is that the reranker was trained on web search (MS MARCO) and favours chunks that repeat the query's words over chunks that prove the skill. H-B4 supported: V8 returns no evidence for 100% of poor-fit requirements and 2.2% of good-fit ones, so it is shipped for the Fit Matcher, where refusing is the point.

Shipped choice. V0 for the Writer's evidence (pre-registered rule: simplest variant within 2 points of the best, p95 under 2 s), V8 for the fit matrix. The embedding model changed once: `nomic-embed-text-v1.5` gave V0 0.947 (`rag_eval_nomic.json`), but it does not fit in the free host's 512 MB, so we measured `bge-small` and accepted 4.9 fewer points (`docs/adr/002-hosting.md`).

Claim verification. Each sentence that makes a claim is checked against the evidence with the reranker (threshold -2.0) plus a tool vocabulary check; unsupported sentences are removed before the Critic sees the draft. Unsupported claims per proposal fell from 0.152 to 0 on 33 proposals.

Where better retrieval did not mean more replies. The ML engineer persona has the best Recall@5 (0.958) but the most unsupported claims before verification (0.364 per proposal) and the same reply rate as the web developer (63.6%), whose recall is 0.861. The data analyst has middling recall and the lowest reply rate (54.5%). In R&D question C the supervisor sent proposals with 1.29 unsupported claims each and still got 85.7% replies. GigSim's clients reward a specific opening and concrete numbers; they do not check claims against a portfolio. Grounding protects the freelancer's reputation, which a reply rate cannot measure.

Judge calibration. We labelled 30 proposals ourselves (`evals/datasets/judge_labels.csv`). Cohen's kappa with the gpt-oss-20b judge is 0.233 (19 of 30 agree). The judge separates good from generic proposals (4.6 vs 1.0) but scored the raw drafts *before* claim verification 5.0, higher than the final drafts. It cannot see grounding, so the judge score is reported but never used to make a decision; human labels and claim verification are the trusted measures.

5. R&D question C: one agent, a prebuilt supervisor, or our graph

Setup. 12 dev jobs (4 per persona, 3 scams), gpt-oss-120b for every system, the same MCP tools. (a) one ReAct agent with all read tools; (b) `langgraph-supervisor` with a researcher and a writer; (c) the GigBid graph. The harness approved every bid so GigSim could reply.

System	Bid or skip accuracy	Scams skipped	Unsupported claims / proposal	Reply rate	Tokens / job
One ReAct agent	0.75	3/3	0.50	66.7%	3,799
Prebuilt supervisor	0.833	3/3	1.29	85.7% (6 of 7)	2,231 (undercounted)
GigBid graph	**1.00**	3/3	**0**	66.7%	6,062

Source: `rnd_c_compare.json`.

Conclusion. The graph is the only system that made every decision correctly and never sent an unsupported claim; it costs about 1.6 times the single agent's tokens, spent on claim checking and the Critic. The supervisor's reply rate rests on 7 proposals and is not a meaningful difference. Its token count only includes messages left in the final state, so its true cost is higher. Two honest notes: the first two supervisor runs failed on every job because `create_supervisor` drops the structured answer unless a state schema is passed (our harness bug, fixed in commit `d5f4419`); and gpt-oss sometimes wrote malformed tool names (`get_policy<|channel|>commentary`) inside the supervisor, so one job needed 3 attempts. Our graph avoids free-form tool loops in its LLM nodes.

6. The change request

What changed. The platform policy moved to version `v2-2026-11-02`: every proposal costs 2 extra Connects and must include the sentence "This proposal was drafted with AI assistance and reviewed by me."

What we edited. One file, `gigsim/policy.yaml` (6 lines added, 3 changed), commit `1cbd92b`; no code. Pricing reads the extra Connects from `get_policy`, and the Compliance Guard reads `required_statements` from the same policy. On 3 jobs the charge rose by exactly 2 Connects and the statement was present every time, added by Compliance, never by the Writer (`change_request.json`, commit `2033e19`, about 40 minutes after the policy commit including the run).

What it says about the design. Platform rules live in data, read through one MCP tool, so a rule change is a configuration change. The change also exposed a gap: the claim checker read "reviewed by me" as an unsupported claim, which the CI smoke eval caught. The fix (commit `2ca416c`) exempts policy-required sentences, still read from the same policy file.

7. Signature extension X2: proposal A/B test

Two writing strategies, each a versioned prompt chosen by configuration, everything else identical: A opens with the client's problem; B is shorter and opens with one sharp question. 30 good-fit dev jobs per arm, all personas, outcomes from GigSim's simulated clients.

Arm	Replied / sent	Reply rate	Average words
A (control)	18 / 30	60%	146
B (question-led)	20 / 29	69%	126

Two-proportion z-test: z = 0.72, p = 0.47, not significant (`ab_test.json`). Detecting a 9-point lift with 80% power would need about 450 proposals per arm. A stays the default and the harness is ready for a larger run (`docs/EXTENSION.md`).

8. Failure analysis

Failures come from `decision_eval.json` (51 jobs, labels in `evals/datasets/dev_labels.json`) and from our 30 human labels (P-numbers in `judge_labels.csv`).

Category	Jobs	What went wrong	Fix tried	Worked?
Wrong skip	job-014, job-029, job-046	Legitimate jobs with a planted injection were flagged high risk and skipped	None: skipping a post that tries to steer the agent is the cautious choice	Accepted; costs 3 of 36 bids (recall 0.917)
Wrong scam flag	job-019	A low-budget but honest post was called a scam by the LLM opinion	None yet; rules alone would not flag it	No
Mispriced	job-001, -017, -033, -036, -042	All vague posts; price 1% to 9% above the fair band	None yet; cause not yet investigated	No
Unsupported claim	job-008, -012, -015, -023	Writer invented details (5 sentences)	Claim verification removes them	Yes, 0 after
Unsupported claim	job-045 (P30)	Writer turned a checkout experiment into a pricing one and claimed SciPy	Tool vocabulary extended, commit `93e3132`	Yes, regression test
Generic or weak answer	job-008 (P05), job-023 (P12)	Checker treated requests ("please share...") and plans as claims and cut them	`REQUEST` and `PLAN` patterns, commit `93e3132`	Yes
Compliance miss (false positive)	job-025 (P01), job-023 (P12)	"Guarantee" rule removed a true sentence and left a client review with no project	Narrowed the rule, commit `745f532`	Yes
Compliance miss (false negative)	job-028 (P09), job-044 (P14), job-015 (P18, P23)	Over-promises ("under two seconds", "match to the cent", "never fabricates")	Guarantee rule targets fixed phrases only	No, still gets through
Deadline conflict	job-031 (P16)	Proposal allowed two weeks on a 7-day job	Deadline passed to the Writer, prompt v4, commit `93e3132`	Yes
Weak first draft	job-008, -010, -012, -015, -017, -022, -023, -035, -043, -048	Critic sent the draft back twice, the maximum (generic opening, removed claims, missing screening answer)	Bounded loop; approval screen shows the Critic's last notes	Partly: costs tokens and time
Tool error	job-006, job-009, job-012 (first run)	Our own Marketplace MCP rate limiter returned `RATE_LIMITED`; jobs dead-lettered	Client waits and retries on `RATE_LIMITED`; `--rerun-failed`	Yes, 0 errors in the final run
Injection	6 injection jobs	No attack succeeded (section 9)	n/a	n/a

Taxonomy summary. 25 distinct jobs failed in at least one way. The largest group is weak first drafts (10), then mispricing (5) and unsupported claims (5). Fixed: unsupported claims, removed requests, the guarantee false positive, the deadline conflict and rate-limit errors. Open: over-promises, vague post pricing, and the LLM's scam opinion on honest cheap posts.

9. Safety and platform respect

What GigBid must never do, and how it is enforced.

- **Send without a human.** Approval is a graph `interrupt()` stored in the checkpointer, not a prompt instruction; submit and send tools run only after resume. The dry-run preview shows the exact payload and Connects cost first. Idempotency keys make a double click or a retry submit once (integration test `test_good_job_pauses_for_approval_then_submits_once`).
- **Break platform rules.** The Compliance Guard removes contact details, off-platform requests and guarantees, and adds required statements; an edited draft is re-checked and blocked until fixed.
- **Invent experience.** Claim verification removes unsupported sentences (0 after verification).
- **Act like a scraper.** Scouting runs every 2 hours on GigSim and 6 hours live; the Marketplace MCP allows 40 calls a minute per user, under GigSim's 60; 429s are waited out, never hammered. The Upwork adapter is a stub; no live account was used.
- **Leak or overreach.** MCP servers require a bearer token; each agent gets an allowlist of tools (`agent/mcp_client.py`); the audit log masks emails, card numbers and phone numbers (`mcp_servers/common.py`); secrets live only in the host's environment.

- **Obey injected text.** Job posts and client messages are wrapped as untrusted data, the Job Analyst flags instructions inside them, and flagged posts raise the risk score.

Measured. Attack success on 6 injection jobs and 5 client-message trials per mode: 0% with defences on and 0% with them off (`injection_eval.json`). gpt-oss ignored these attacks on its own, so this experiment does not show that the defences are needed; it shows they cost 3 legitimate bids.

What still gets through. Over-promises phrased in new words (P09, P14, P18, P23), and claims that are true for the portfolio but misattributed between projects, which the reranker cannot tell apart.

10. Hidden test results against the declared budgets

The hidden test run happens after submission, so its results are not in this report. What we can state now, without excuses:

- **Dev budgets** (section 1): every declared budget was met on the 51 dev jobs. The p95 latency (116.5 s) is close to its 120 s budget, and the ML persona alone was over it (169.6 s p95); it also had the most Critic revisions.
- **Hosting.** Render free, Singapore region, one process (GigSim and the MCP servers in process), peak memory measured at 441 MB of 512 MB. Graph runs are processed one at a time on the host, so a burst of new jobs is analysed in sequence, about 20 s to 2 minutes each.
- **Speed.** API calls take 0.14 to 0.27 s after moving the service next to the database (commit `85b1a2b`, plus a connection pool in `b2a2c08`); in the first region they took 0.7 to 1.8 s.
- **Risks for the hidden run.** The free service sleeps after 15 minutes idle and takes about a minute to wake; the free model quota (200,000 tokens a day for gpt-oss-120b) can run out under a large batch, in which case calls fall back to gpt-oss-20b and wait for quota instead of failing.

Appendix A: reproducing every number

```
pip install -e ".[dev]"                          # Python 3.11 or later
python -m evals.runners.rag_eval                 # rag_eval.json (section 4)
python -m evals.runners.decision_eval            # decision_eval.json (sections 1, 4, 8)
python -m evals.runners.rnd_a_tools              # rnd_a_tools.json (section 3)
python -m evals.runners.rnd_c_compare            # rnd_c_compare.json (section 5)
python -m evals.runners.change_request           # change_request.json (section 6)
python -m evals.runners.ab_test                  # ab_test.json (section 7)
python -m evals.runners.injection_eval           # injection_eval.json (section 9)
python -m evals.runners.judge_calibration        # judge_calibration.json (section 4)
python scripts/make_traces.py                    # trace_links.json (public traces)
```

Appendix B: public traces

1. Scam skipped before any writing: https://eu.smith.langchain.com/public/9658d6fb-b18d-484b-ad71-2329b764e776/r
2. Critic sends a draft back once, then the graph pauses for approval: https://eu.smith.langchain.com/public/0ab2f0fc-ca53-4a62-9aca-d37c584b724e/r
3. After a restart, a new runtime resumes the paused job and submits once: https://eu.smith.langchain.com/public/b6c10839-f9be-4e47-be7a-cfea81347f75/r